Smart Patient Control System

Lovepreet Sidhu	Jack Xu	Jackson Maclean

INFO 4381 – Internet of Things

Al-Ani, Mayyadah

April 5, 2026

Table of Contents

Introduction

Background information on IoT and its significance

The Internet of Things (IoT) refers to interconnected physical devices that collect, exchange, and analyze data through sensors and networks. IoT plays a critical role in modern healthcare by enabling real-time monitoring, improving patient safety, and reducing manual intervention. Smart healthcare systems help in early detection of critical conditions and provide timely alerts to caregivers.

Overview of the specific problem being addressed

In traditional healthcare environments, continuous monitoring of patient vitals requires constant human supervision. This can lead to delayed responses in emergencies. The problem addressed in this project is the lack of an automated, low-cost system that can monitor patient conditions and trigger alerts when abnormal conditions occur.

Objectives of the project

- Design an IoT-based patient monitoring system
- Monitor temperature, motion, and simulated heart rate
- Classify conditions into Normal, Caution, and Critical states
- Trigger alerts using LEDs, buzzer, and display
- Implement nurse call and acknowledgment system

Scope of the report

This report covers the design, implementation, and evaluation of a prototype IoT-based patient monitoring system using Arduino and Tinkercad simulation.

System Design

Description of the overall system architecture

The system consists of multiple sensors connected to an Arduino microcontroller, which acts as the central processing unit. The Arduino collects real-time data from sensors, analyzes it using predefined thresholds, and controls output devices such as LEDs, buzzer, servo motor, and LCD display. The system follows an edge computing model, where all processing is done locally without requiring cloud connectivity.

IoT 7-Layer Architecture Mapping

The Smart Patient Control System can be mapped to the standard 7-layer IoT architecture to better explain how data flows through the system and how each component contributes to overall functionality.

- **Layer 1 – Physical Devices:**

This layer includes all sensors and actuators used in the system. The temperature sensor, potentiometer (for BPM simulation), and PIR motion sensor collect real-world data, while the RGB LED, buzzer, and servo motor act as output devices to respond to system conditions.

Future technologies: Real heart rate sensors (e.g., MAX30100), wearable devices, medical-grade sensors.

- **Layer 2 – Connectivity:**

Communication between components is handled through wired connections on the Arduino board. Protocols such as I2C are used for LCD communication, and UART is used for serial monitoring.

Future technologies: Wi-Fi (ESP8266/ESP32), Bluetooth Low Energy (BLE), MQTT protocol for cloud communication.

- **Layer 3 – Edge Computing:**

The Arduino Uno acts as the edge device, where all data processing occurs locally. It reads sensor values, applies threshold-based logic, and determines system states (Normal, Caution, Critical) without relying on cloud services.

Future technologies: Raspberry Pi, NVIDIA Jetson Nano for advanced edge analytics and AI-based health monitoring.

- **Layer 4 – Data Accumulation:**

Currently, the system does not store data persistently. However, serial output acts as temporary data logging.

Future technologies: Cloud storage using AWS IoT Core, Azure IoT Hub, Firebase, or local databases like SQLite for storing patient history.

- **Layer 5 – Data Abstraction:**

Data is processed and organized using rule-based logic in the code. Sensor inputs are converted into meaningful categories (Normal, Caution, Critical), making it easier for the system to interpret and act on the data.

Future technologies: Data pipelines, ETL tools, and APIs for integrating with hospital management systems.

In future, this layer can be improved using data pipelines and APIs. Instead of only classifying data instantly, the system can send data to the cloud where it is cleaned, stored, and structured. For example, patient data can be tracked over time to identify trends such as gradually increasing temperature. APIs can also allow this processed data to be shared with hospital systems, enabling better monitoring, reporting, and decision-making.

- **Layer 6 – Applications:**

The application layer is represented by the LCD display and user interface, where real-time system status is shown.

Future technologies: Mobile apps, web dashboards, real-time monitoring systems for doctors and caregivers.
Currently, the application layer is limited to an LCD display that shows real-time system status (Normal, Caution, Critical). The user must be physically present to view the data.

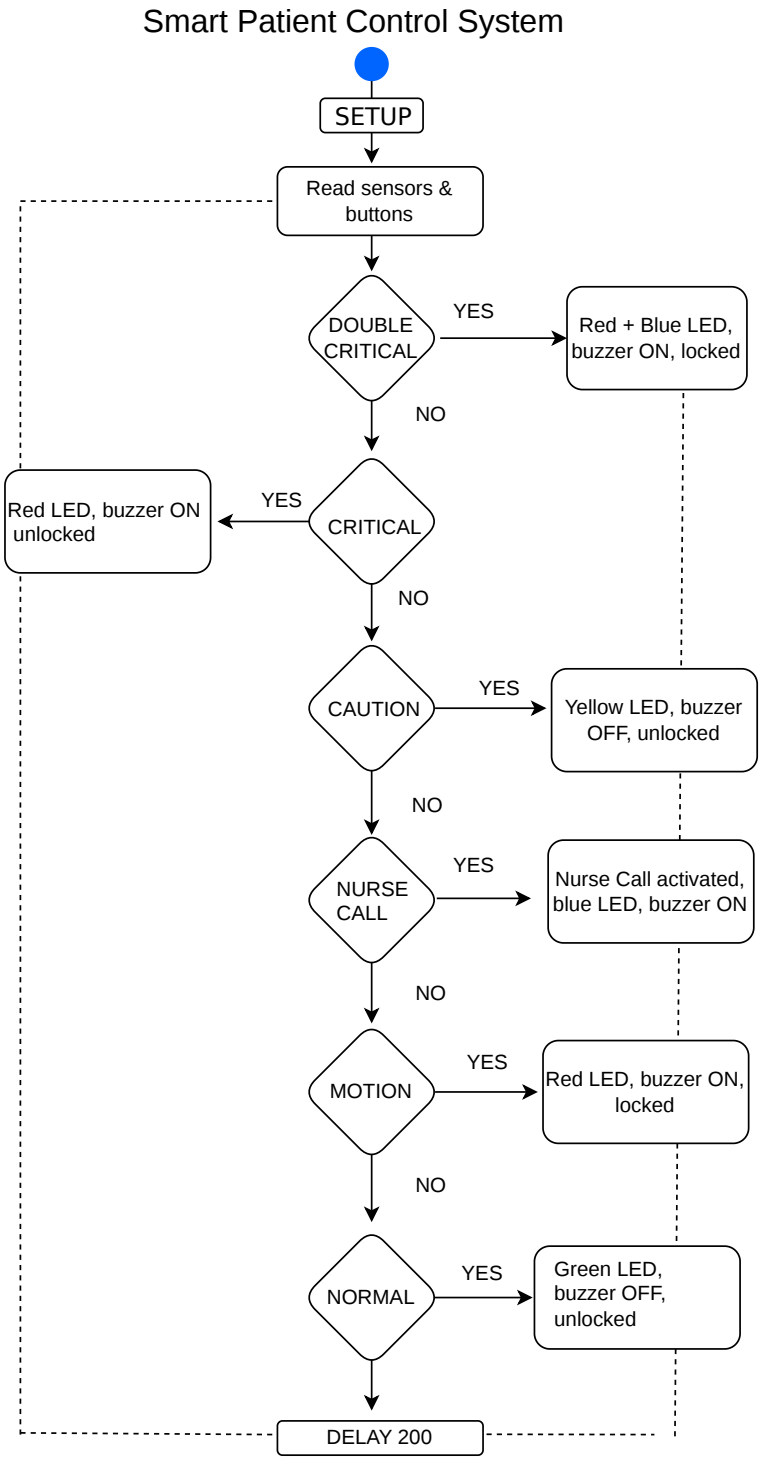
In the future, this layer will include mobile apps and web dashboards for remote monitoring. Sensor data will be sent to a backend system, which will provide real-time updates to these applications. For example, doctors can monitor patient vitals through a mobile app, receive alerts when conditions become critical, and view data trends using graphs. This improves accessibility and real-time monitoring.

- **Layer 7 – Collaboration & Processes:**

This layer includes system actions such as triggering alarms, locking mechanisms, and nurse call functionality. These processes ensure timely response to critical conditions and simulate real-world healthcare workflows.

Future technologies: Automated alert systems (SMS/Email), integration with hospital workflows, AI-based decision support systems.

(diagram)


Smart Patient Control System
SETUP
Read sensors & buttons
DOUBLE CRITICAL
YES
Red + Blue LED, buzzer ON, locked
NO
CRITICAL
YES
Red LED, buzzer ON unlocked
NO
CAUTION
YES
Yellow LED, buzzer OFF, unlocked
NO
NURSE CALL
YES
Nurse Call activated, blue LED, buzzer ON
NO
MOTION
YES
Red LED, buzzer ON, locked
NO
NORMAL
YES
Green LED, buzzer OFF, unlocked
DELAY 200

Components

Sensors used and their type

- Temperature Sensor (Analog)
- Potentiometer (Analog – simulates heart rate)
- PIR Motion Sensor (Digital) detects patient movement

Actuators used and their type

- RGB LED (Digital Output) status indication
- Buzzer (Digital Output) alarm system
- Servo Motor (PWM Output door/lock control

Other devices involved

- Arduino Uno
- LCD Display (I2C)
- Buttons / IR Remote for control

Type of data being collected

- Temperature (°C)
- Heart rate (BPM simulation)
- Motion detection (Yes/No)

Edge devices and fog nodes

- Edge device: Arduino Uno process data locally
- Fog nodes: Not implemented

Cloud platform (future use)

- AWS IoT Core
- Azure IoT Hub

Protocols used

- I2C (LCD communication)
- UART (Serial debugging)
- MQTT (future)

Types of Data

- Real-time sensor data
- Alert state data (Normal, Caution, Critical)

Data Frequency

- Continuous real-time updates

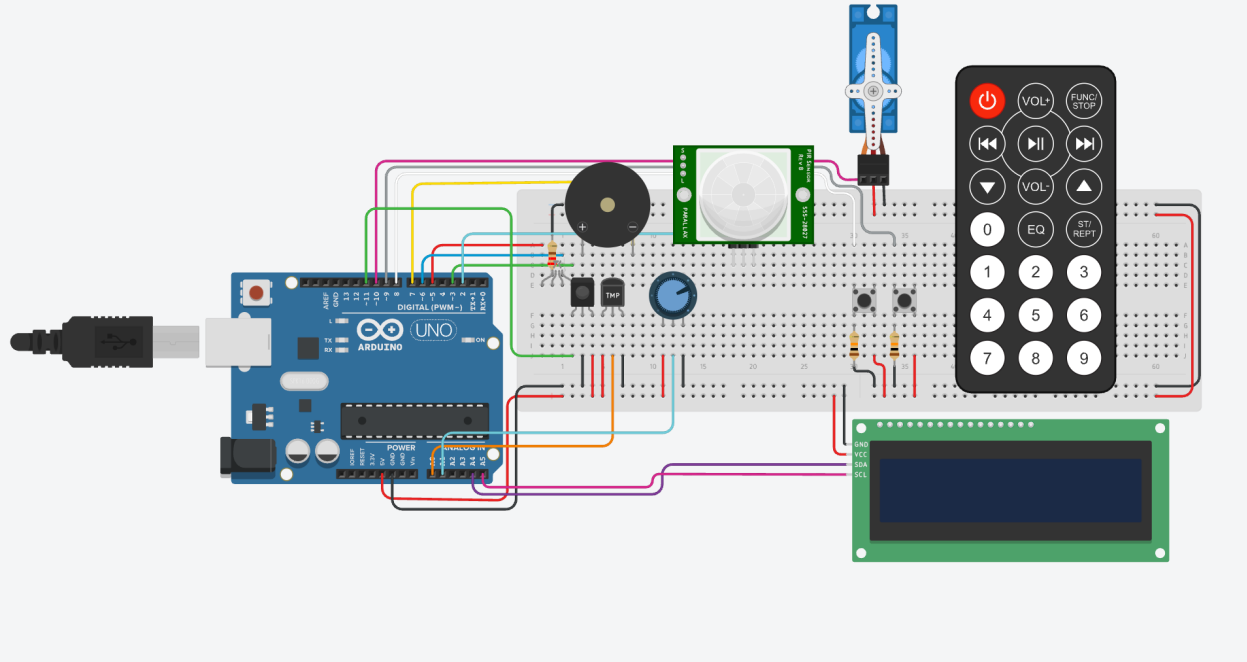
Data Analytics methods

- Threshold-based logic
- Rule-based classification for alert states

Security approach

- Local processing
- Authentication using secure APIs (future)
- Physical isolation for edge device

Screenshot of the circuit



Implementation

Step-by-step explanation

1. Connected sensors to Arduino
2. Integrated LCD using I2C
3. Programmed sensor readings
4. Defined thresholds for system states
5. Configured LEDs, buzzer, and servo
6. Implemented nurse call system
7. Tested in Tinkercad

Simulation link

Link

Connection pins used

Component	Pins & Connection Point
I2C LCD Display	A4 (SDA), A5 (SCL)
RGB LED	3 (Green), 5 (Red), 6 (Blue)
Buzzer	7 (Negative)
PIR Sensor	2 (Signal)
Temperature Sensor	A0 (Vout)
Potentiometer (Heartbeat Sensor)	A1 (Wiper)
IR Sensor	11 (Out)
Servo Motor	10 (Signal)
Button (Ack/reset)	8 (Terminal 1b)
Button (Mode)	9 (Terminal 1b)

Code (with explanation)

Refer to Appendix for code

The system code involves importing the required libraries, such as #include <LiquidCrystal_I2C.h> for communication with the LCD, #include <Wire.h>, #include <Servo.h> for the servo motor behavior used in the lock system, and #include <IRremote.h>, which is partially implemented due to issues like invalid header file errors. Therefore, we relied on buttons instead.

Later, we defined the pins for the sensors, actuators, and other devices involved, such as the temperature sensor, IR remote, potentiometer, motion detector, and RGB LED pins. The code also defines pins for remaining devices like the servo motor, and ACK and Mode buttons, which are used for the nurse calling feature of the program.

After that, we implemented IR remote button codes, initialized the servo motor and LCD with values (0x27, 16, 2) for the I2C LCD, and defined boolean variables such as isSilenced for sound control, isUnlocked for the lock system, and nurseAct, setting them to false to initialize the system. We also implemented lastMode and lastAck boolean values and set them to LOW so that button presses can be detected properly.

In the setup() function, we used Serial.begin(9600) for serial output and configured all pins using pinMode for inputs (sensors and buttons) and outputs (RGB LED and buzzer). The servo was

initialized using serv.attach(servoPin) and set to angle 0. The LCD was initialized with init() and backlight(), and displayed “System Ready” and “Monitoring…”.

After that, we used variables like float voltage = analogRead(tempPin) to calculate temperature, bpm = map(analogRead(potPin), 0, 1023, 0, 300) for heart rate, and motion = digitalRead(pirPin) to gather sensor data. Later in the main loop, we defined boolean conditions for monitoring such as tempCrit, bpmCrit, tempCaut, bpmCaut, tempnorm, bpmnorm, and allNormal.

Then for buttons, we used bool modeState = digitalRead(modePin) and bool ackState = digitalRead(ackPin). If the Mode button is pressed while all conditions are normal, nurseAct becomes true. If any critical condition occurs, nurseAct is turned off. If the ACK button is pressed, the system is silenced and unlocked. The lastMode and lastAck variables are updated to track button state changes.

The IR remote logic (currently commented out) was intended to handle Mode and ACK actions using remote button inputs, but was not used due to implementation issues.

Later, we used lcd.setCursor and lcd.print to display temperature and bpm on LCD line 1, and also used Serial.print to show the same values on the serial monitor for checking and debugging purposes. And then we used lcd.setCursor for line 2, cleared the previous text, and prepared it to display system states like double critical and other conditions.

This section is supported by our helper functions, where buzz() controls the buzzer based on silence state and frequency, setColor() controls the RGB LED using analog values, and lock() controls the servo motor to lock (90°) or unlock (0°) the system.

Later, we implemented the main condition logic where the system checks double critical first (temp and bpm both critical) and shows "CRIT TB" with flashing red and blue LED, buzzer, and lock. Then it checks individual critical conditions for temperature and bpm and shows "CRIT T" or "CRIT B" with red LED and buzzer. After that, caution conditions are checked and display "CAUT T" or "CAUT B" with yellow LED and no buzzer. Then nurse call is handled via mode button when everything is notmal, when activated, showing "NURSE CALL" with blue LED and buzzer. Motion is also checked to lock the system with red LED and alert. Finally, if none of the conditions are true, the system goes to normal state showing "NORMAL" with green LED, no buzzer, and unlocked state.

Challenges faced

- Tinkercad compatibility issues with IR remote library
- Memory limitations of Arduino Uno
- LCD flickering

Solutions:

- Tested IR separately

- Optimized code usage
- Reduced LCD refresh rate

Testing and Results

Testing methodology

- Simulated various sensor inputs
- Tested all states (Normal, Caution, Critical, Double Critical)

Results obtained

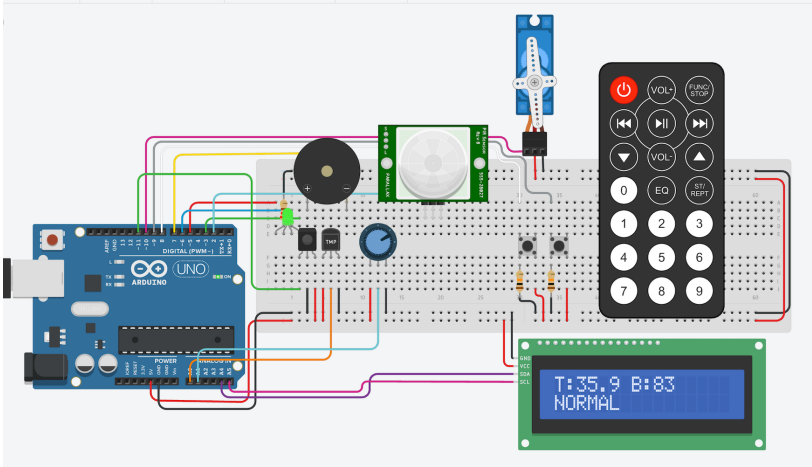
Test Case Table

Test Case	Input Condition (Temp, BPM, Motion)	Expected condition	Expected Output	Actual Output	Result
Normal	T = 35.9°C, B = 83	tempinC >= 35.0 && tempinC <= 38.5 && bpm >= 50 && bpm <= 110	Green LED, buzzer OFF, unlocked	Same	Pass
Temp Critical	T = 53°C, B = 83	tempinC > 38.5 || tempinC < 35.0	Red LED, buzzer ON, unlocked	Same	Pass
BPM Critical	T = 35.9°C, B = 24	bpm > 130 || bpm < 50	Red LED, buzzer ON, unlocked	Same	Pass
Double Critical	T = 14°C, B = 24	(tempinC > 38.5 || tempinC < 35.0) && (bpm > 130 || bpm < 50)	Red + Blue LED, buzzer ON, locked	Same	Pass
Temp Caution	T = 37.9°C, B = 90	tempinC >= 37.6 && tempinC <= 38.5	Yellow LED, buzzer OFF, unlocked	Same	Pass
BPM Caution	T = 35.9°C, B = 119	bpm >= 111 && bpm <= 130	Yellow LED, buzzer OFF, unlocked	Same	Pass

Motion Lock	Motion detected	motion && !isUnlocked	Red LED, buzzer ON, locked	Same	Pass
ACK Button	Pressed during alert	lastAck == LOW && ackState == HIGH	Buzzer OFF, system silenced	Same	Pass
Mode Button	Pressed in normal	lastMode == LOW && modeState == HIGH && allNormal	Nurse Call activated, blue LED, buzzer ON	Same	Pass

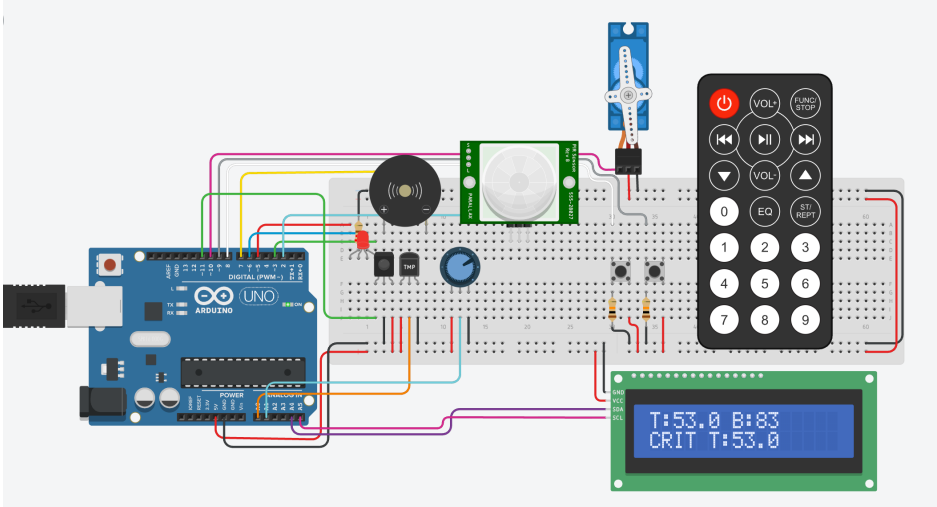
I. Normal

Normal condition Temp in normal range BPM in normal range No motion
Expected: LCD shows NORMAL, green LED, buzzer off, servo unlocked



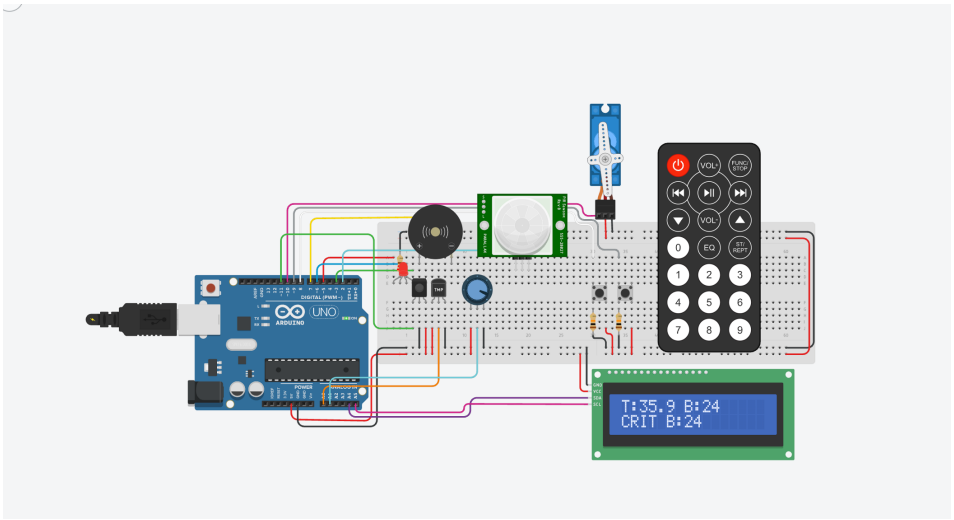
II. Temperature critical

Temp > 38.5 or < 35.0
BPM normal
Expected: LCD shows critical temp message, red LED, buzzer on, servo unlocked



III. BPM critical

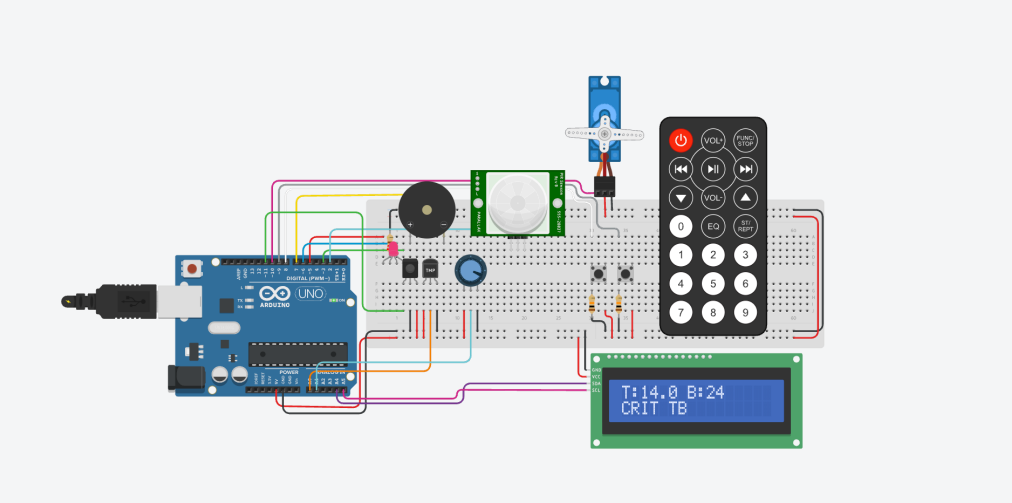
BPM critical BPM > 130 or < 50 Temp normal
Expected: LCD shows critical BPM message, red LED, buzzer on, servo unlocked



IV. Double critical

Temp critical and BPM critical together

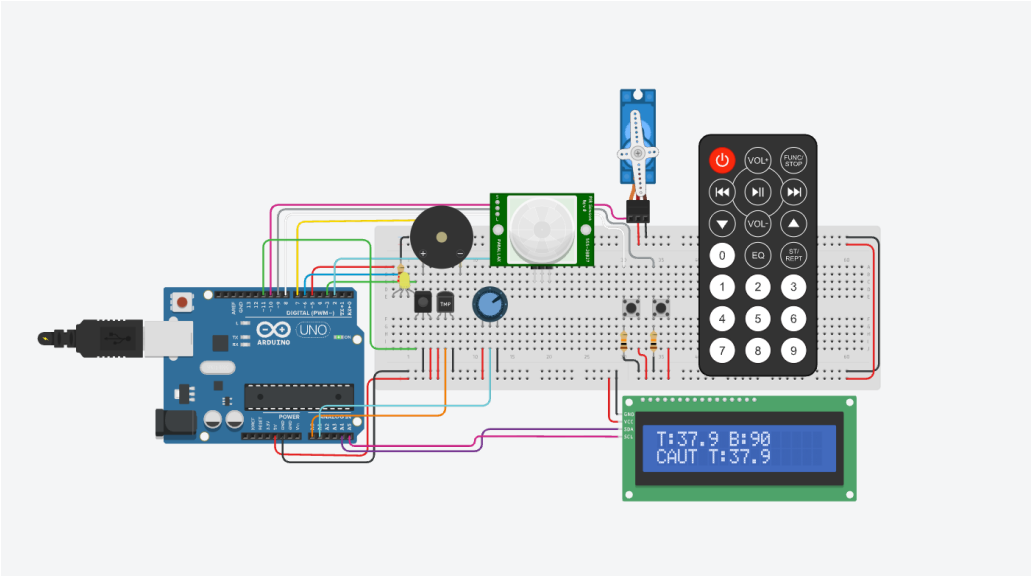
Expected: LCD shows CRIT TB, flashing red/blue behavior, buzzer on, servo locked



V. Caution Temp

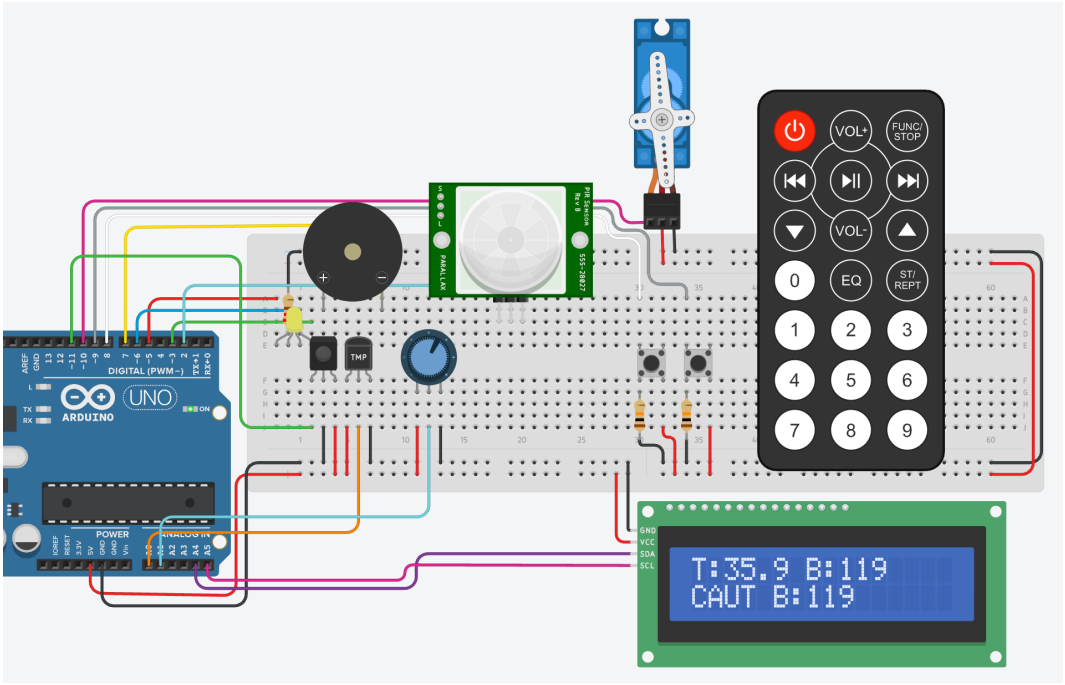
Temp between 37.6 and 38.5 BPM normal

Expected: LCD shows caution temp, yellow LED, buzzer off



VI. Caution BPM

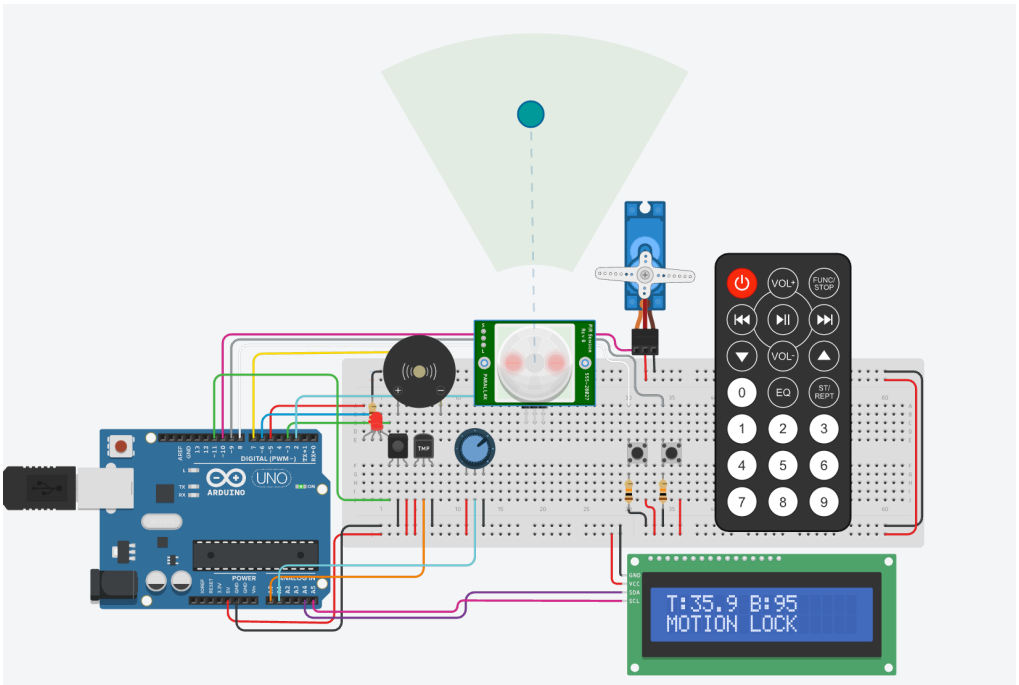
Expected: LCD shows caution temp, yellow LED, buzzer off



VII. Motion detected

while locked Motion = true isUnlocked = false

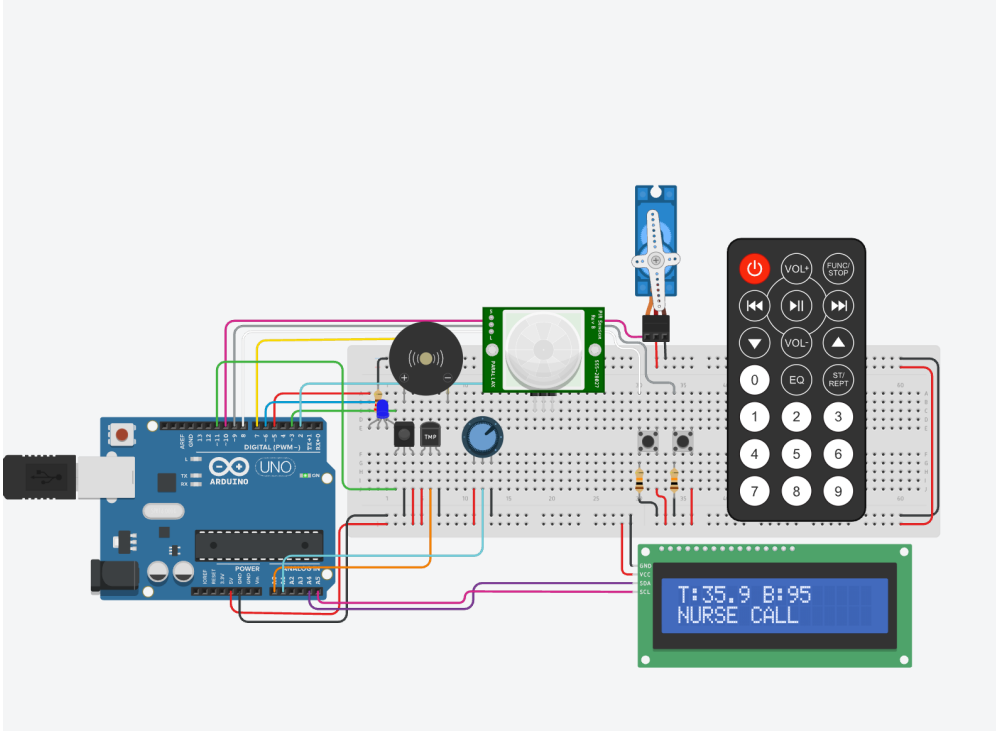
Expected: LCD shows MOTION LOCK, red LED, buzzer on, servo locked



VIII. Nurse call: Mode button

Press Mode while all values are normal

Expected: nurseAct = true, LCD shows NURSE CALL, blue LED, buzzer on and ack pressed silenced the nurse call and system went into normal state.



Analysis

The system successfully detects abnormal conditions and triggers appropriate alerts. It demonstrates the effectiveness of IoT in real-time monitoring.

Limitations and future work

Limitations:

- No real cloud integration (system works fully offline)
- Sensors are simulated (potentiometer instead of real heart rate sensor)
- Limited accuracy due to basic components and simulation environment
- No data storage or historical tracking of patient data
- No remote monitoring capability for doctors/caregivers

Future Work:

- Integrate cloud platforms (e.g., AWS IoT Core or Azure IoT Hub) for real-time data storage and access
- Replace simulated inputs with real medical sensors
- Develop a mobile or web application for remote patient monitoring
- Add wireless communication (Wi-Fi/Bluetooth using ESP32)
- Implement data analytics or AI for predictive health monitoring
- Enable automated alerts (SMS/email) for emergency situations

Conclusion

Summary

The project successfully developed a smart IoT-based patient monitoring system capable of detecting abnormal conditions and triggering alerts.

Final thoughts

This system demonstrates the potential of IoT in healthcare by enabling real-time monitoring and automated responses to patient conditions. It provides a simple and effective solution for detecting abnormal states and alerting caregivers. However, the current implementation mainly focuses on the core layers of the Cisco IoT reference model. With further improvements such as cloud integration, data storage, and mobile applications, the system can be expanded to cover all layers and become a more scalable and fully functional smart healthcare solution.

References

Amazon. (2022). *IOT Core Documentation* . Amazon.com. https://docs.aws.amazon.com/iot/

Arduino. (n.d.). *Arduino Docs | Arduino Documentation*. Docs.arduino.cc. https://docs.arduino.cc/

Baker, S. B., Xiang, W., & Atkinson, I. (2017). Internet of Things for Smart Healthcare: Technologies, Challenges, and Opportunities. *IEEE Access*, *5*, 26521–26544. https://doi.org/10.1109/access.2017.2775180

Cisco. (2014). *The Internet of Things Reference Model*. Cisco Systems. https://www.cisco.com

Hu, F., Xie, D., & Shen, S. (2013, August 1). *On the Application of the Internet of Things in the Field of Medical and Health Care*. IEEE Xplore. https://doi.org/10.1109/GreenCom-iThings-CPSCom.2013.384

Sun, W., Cai, Z., Li, Y., Liu, F., Fang, S., & Wang, G. (2018). Security and Privacy in the Medical Internet of Things: A Review. *Security and Communication Networks*, *2018*, 1–9. https://doi.org/10.1155/2018/5978636

Tinkercad. (2019). *Tinkercad Circuits*. Tinkercad. https://www.tinkercad.com

Appendices

Appendix A

```
#include <LiquidCrystal_I2C.h>

#include <Wire.h>

#include <Servo.h>

#include <IRremote.h>


// Pins

const int tempPin = A0; // Tempurature Sensor (Analog Input)

const int IRPIN = 11; //ir

const int potPin = A1; // Potentiometer Pin (Analog Input)

const int pirPin = 2; // Motion Detector Pin (Digital Input)

const int redPin = 5; // Red LED (Digital Output)

const int bluePin = 6; // Blue LED (Digital Output)

const int greenPin = 3; // Green LED (Digital Output)

const int buzzPin = 7; // Buzzer (Digital Ouput)

const int servoPin = 10; // Servo

const int ackPin = 8; //Ack/Reset Button

const int modePin = 9; //Mode Button

// IR remote button codes
const int BUTTON0_Ack = 0x0C;
const int BUTTON1_Mode = 0x10;
const int BUTTON2 = 0x11;
```

```
const int BUTTON3 = 0x12;
const int BUTTON4 = 0x14;
const int BUTTON5 = 0x15;
const int BUTTON6 = 0x16;
const int BUTTON7 = 0x18;
const int BUTTON8 = 0x19;
const int BUTTON9 = 0x1A;

Servo serv;
LiquidCrystal_I2C lcd(0x27, 16, 2);

bool isSilenced = false;
bool isUnlocked = false;
bool nurseAct = false;

bool lastMode = LOW;
bool lastAck = LOW;

void setup() {
  Serial.begin(9600);

  pinMode(tempPin, INPUT);
  pinMode(potPin, INPUT);
  pinMode(pirPin, INPUT);

  pinMode(redPin, OUTPUT);
  pinMode(bluePin, OUTPUT);
  pinMode(greenPin, OUTPUT);
  pinMode(buzzPin, OUTPUT);

  pinMode(ackPin, INPUT);
  pinMode(modePin, INPUT);

  serv.attach(servoPin);
  serv.write(0);

  lcd.init();
  lcd.backlight();

  lcd.setCursor(0, 0);
```

```
  lcd.print("System Ready");
  lcd.setCursor(0, 1);
  lcd.print("Monitoring...");
  delay(1000);
  lcd.clear();

  // Start IR receiver
  //IrReceiver.begin(IRPIN, ENABLE_LED_FEEDBACK);
}

void loop() {

  // ===== SENSOR DATA =====
  float voltage = analogRead(tempPin) * (5.0 / 1024.0);
  float tempinC = (voltage - 0.5) * 100;

  int bpm = map(analogRead(potPin), 0, 1023, 0, 300);

  bool motion = digitalRead(pirPin);

  // ===== CONDITIONS =====
  bool tempCrit = (tempinC > 38.5 || tempinC < 35.0);
  bool bpmCrit = (bpm > 130 || bpm < 50);

  bool tempCaut = (tempinC >= 37.6 && tempinC <= 38.5);
  bool bpmCaut = (bpm >= 111 && bpm <= 130);

  bool tempnorm = (tempinC >= 35.0 && tempinC <= 38.5);
  bool bpmnorm = (bpm <= 110 && bpm >= 50);
  bool allNormal = tempnorm && bpmnorm;

  // ===== BUTTONS =====
  bool modeState = digitalRead(modePin);
  bool ackState = digitalRead(ackPin);

  if (lastMode == LOW && modeState == HIGH && allNormal) {
    nurseAct = true;
  }

  if (tempCrit || bpmCrit) {
```

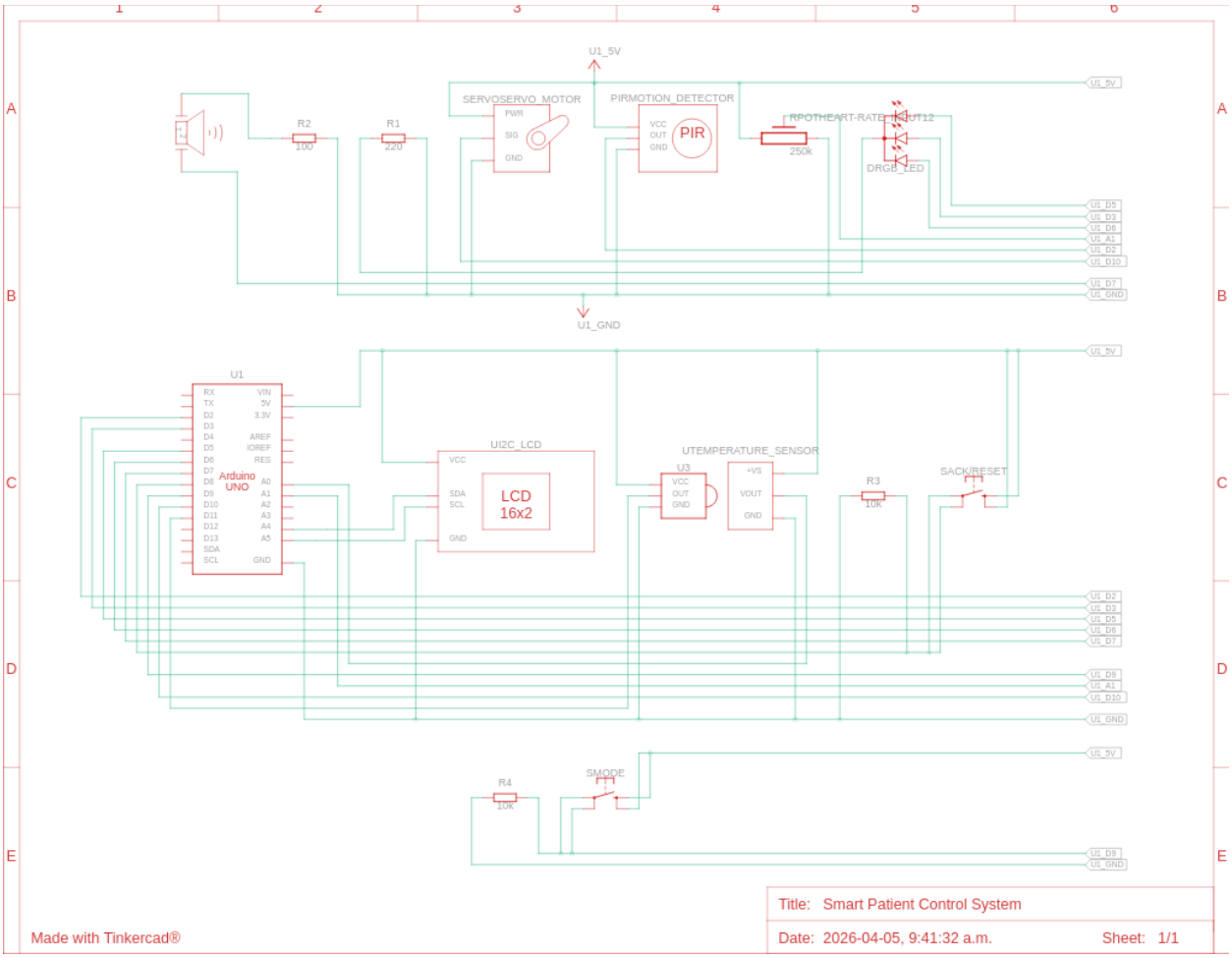
```
  nurseAct = false;
}

if (lastAck == LOW && ackState == HIGH) {
  isSilenced = true;
  isUnlocked = true;
  nurseAct = false;
}

lastMode = modeState;
lastAck = ackState;

// ===== IR REMOTE =====
//if (IrReceiver.decode()) {
//  int cm = IrReceiver.decodedIRData.command;

//  Serial.print("IR Command: 0x");
// Serial.println(cm, HEX);

// switch (cm) {
//  case BUTTON1_Mode:
//   if (allNormal) {
//     nurseAct = true;
//   }
//   break;

// case BUTTON0_Ack:
//  isSilenced = true;
//  isUnlocked = true;
//  nurseAct = false;
//  break;
//  }

//  IrReceiver.resume();
// }

// ===== LCD LINE 1 =====
lcd.setCursor(0, 0);
lcd.print("T:");
lcd.print(tempinC, 1);
```

```
  lcd.print(" B:");
  lcd.print(bpm);
  lcd.print("   ");

  Serial.print("Temp: ");
  Serial.print(tempinC);
  Serial.print(" | BPM: ");
  Serial.println(bpm);

  // ===== LCD LINE 2 =====
  lcd.setCursor(0, 1);
  lcd.print("                ");
  lcd.setCursor(0, 1);

  // --- DOUBLE CRITICAL ---
  if (tempCrit && bpmCrit) {
    lcd.print("CRIT TB");
    setColor(255, 0, 0);
    buzz(1000);

    if ((millis() / 500) % 2 == 0) {
      setColor(0, 0, 255);
    }

    lock(true);
  }

  // --- CRITICAL ---
  else if (tempCrit) {
    lcd.print("CRIT T:");
    lcd.print(tempinC, 1);
    setColor(255, 0, 0);
    buzz(1000);
    lock(false);
  } else if (bpmCrit) {
    lcd.print("CRIT B:");
    lcd.print(bpm);
    setColor(255, 0, 0);
    buzz(1000);
    lock(false);
```

```
  }

  // --- CAUTION ---
  else if (tempCaut) {
    lcd.print("CAUT T:");
    lcd.print(tempinC, 1);
    setColor(255, 255, 0);
    buzz(0);
    lock(false);
  } else if (bpmCaut) {
    lcd.print("CAUT B:");
    lcd.print(bpm);
    setColor(255, 255, 0);
    buzz(0);
    lock(false);
  }

  // --- NURSE CALL ---
  else if (nurseAct) {
    lcd.print("NURSE CALL");
    setColor(0, 0, 255);
    buzz(800);
    lock(false);
  }

  // --- MOTION ---
  else if (motion && !isUnlocked) {
    lcd.print("MOTION LOCK");
    setColor(255, 0, 0);
    buzz(1000);
    lock(true);
  }

  // --- NORMAL ---
  else {
    isSilenced = false;
    isUnlocked = false;
    lcd.print("NORMAL");
    setColor(0, 255, 0);
    buzz(0);
```

```
    lock(false);
  }

  delay(200);
}

// ===== BUZZER =====
void buzz(int hz) {
  if (isSilenced == true || hz == 0) {
    noTone(buzzPin);
  } else {
    tone(buzzPin, hz);
  }
}

// ===== RGB LED =====
void setColor(int redValue, int greenValue, int blueValue) {
  analogWrite(redPin, redValue);
  analogWrite(greenPin, greenValue);
  analogWrite(bluePin, blueValue);
}

// ===== SERVO LOCK =====
void lock(bool val) {
  if (val) serv.write(90);
  else serv.write(0);
}
```

Appendix B


U1_5V
SERVOSERVO_MOTOR
PIRMOTION_DETECTOR
RPOTHEART-RATE-INPUT12
DRGB_LED
R2
100
R1
220
250k
PIR
U1_GND
U1
Arduino UNO
UI2C_LCD
LCD 16x2
UTEMPERATURE_SENSOR
U3
R3
10K
SACK/RESET
R4
10K
SMODE
Made with Tinkercad®
Title: Smart Patient Control System
Date: 2026-04-05, 9:41:32 a.m.
Sheet: 1/1